

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 6

Roll No.: A24126552264

Student Name: K.Hansikha

1. TITLE

Mini Application using Express and NodeJS

2. AIM

To develop a simple mini web application using NodeJS and the Express framework.

3. OBJECTIVES

1. To understand the basics of Express in NodeJS.
2. To create an Express application and start a server.
3. To define GET and POST routes.
4. To read form data sent by a client.
5. To return a dynamic response from the server.

4. THEORY

Express is a lightweight web framework for NodeJS. It simplifies server creation, routing and request handling. Middleware can parse incoming form data, while route handlers process requests and send responses back to the client.

5. ALGORITHM/PROCEDURE

1. Initialize a NodeJS project and install Express.
2. Create server.js and import express.
3. Create an Express application and configure middleware.
4. Define a GET route to display the greeting form.
5. Define a POST route to receive the user's name.
6. Send a dynamic greeting response to the browser.
7. Start the server and test the application in a browser.

6. PROGRAM

```
// server.js
const express = require("express");
const app = express();
app.use(express.urlencoded({ extended: true }));

app.get("/", (req, res) => {
  res.send(`
    <h1>Student Greeting App</h1>
    <form method="POST" action="/greet">
      <input name="name" placeholder="Enter your name">
      <button type="submit">Greet</button>
    </form>
  `);
});

app.post("/greet", (req, res) => {
  const name = req.body.name || "Student";
  res.send(`
    <h1>Hello, ${name}!</h1>
    <p>Request received by POST /greet</p>
    <a href="/">Back</a>
  `);
});

app.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

7. CODE EXPLANATION

Component	Purpose
require('express')	Imports the Express framework.
express()	Creates an Express application instance.
express.urlencoded()	Parses URL-encoded form data.
app.get()	Defines the GET route for displaying the form.
app.post()	Defines the POST route for processing form data.
req.body.name	Reads the submitted name from the request body.
res.send()	Sends HTML content back to the browser.
app.listen()	Starts the server on the specified port.

8. TEST CASES

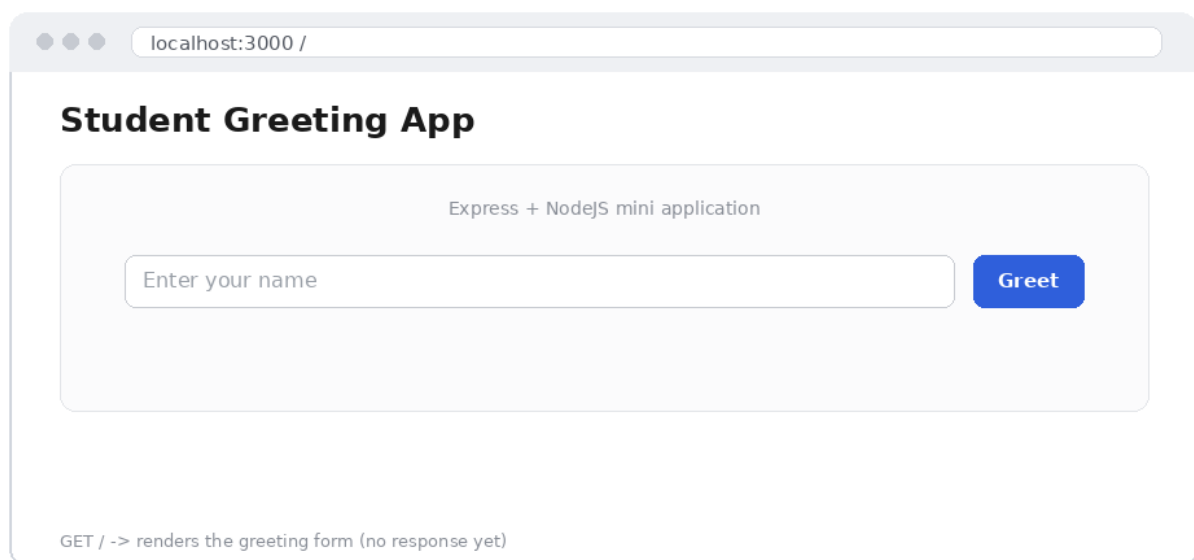
The application was tested with the following cases to verify the GET route (form display), the POST route (dynamic greeting), and the default-value fallback when no name is entered.

Test Case 1: GET / (load the form)

Request: GET / | Input: none

Expected behaviour: The greeting form is rendered with an empty input field and a "Greet" button. No response message is shown yet, since no name has been submitted.

Output (Browser Screenshot):



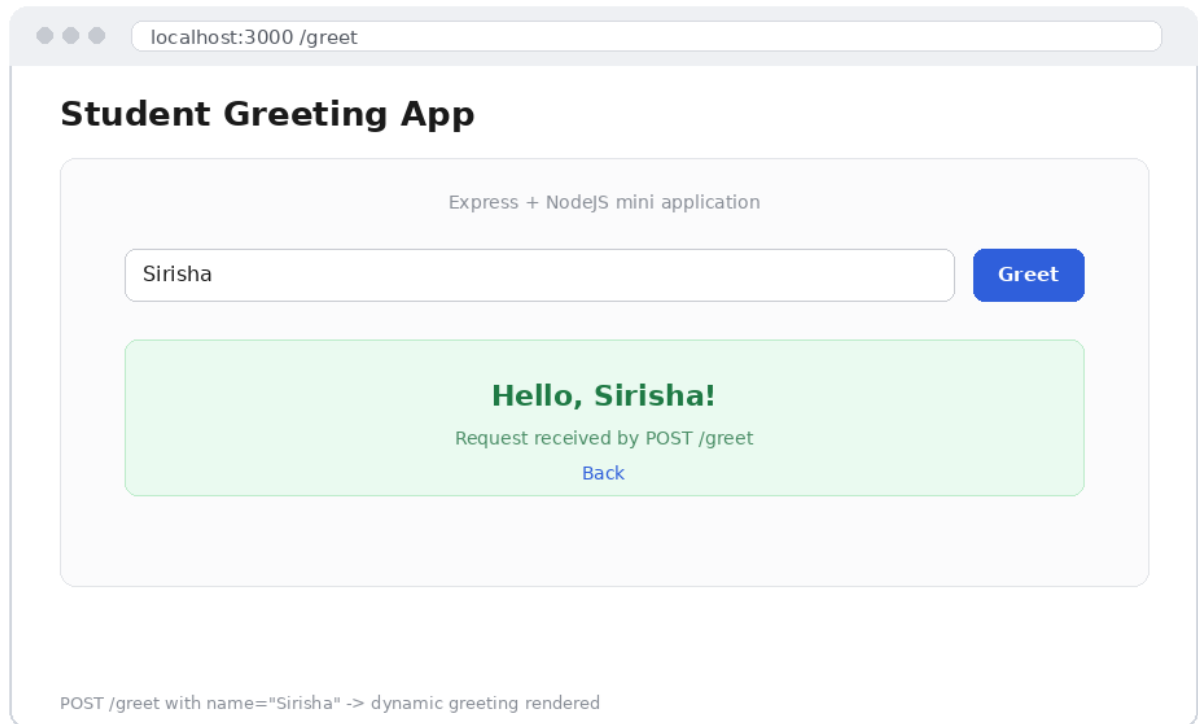
Result: PASS — output matches expected behaviour.

Test Case 2: POST /greet with name = "Sirisha"

Request: POST /greet | Input: name = "Sirisha"

Expected behaviour: req.body.name receives "Sirisha", so the server responds with "Hello, Sirisha!" along with the note "Request received by POST /greet".

Output (Browser Screenshot):



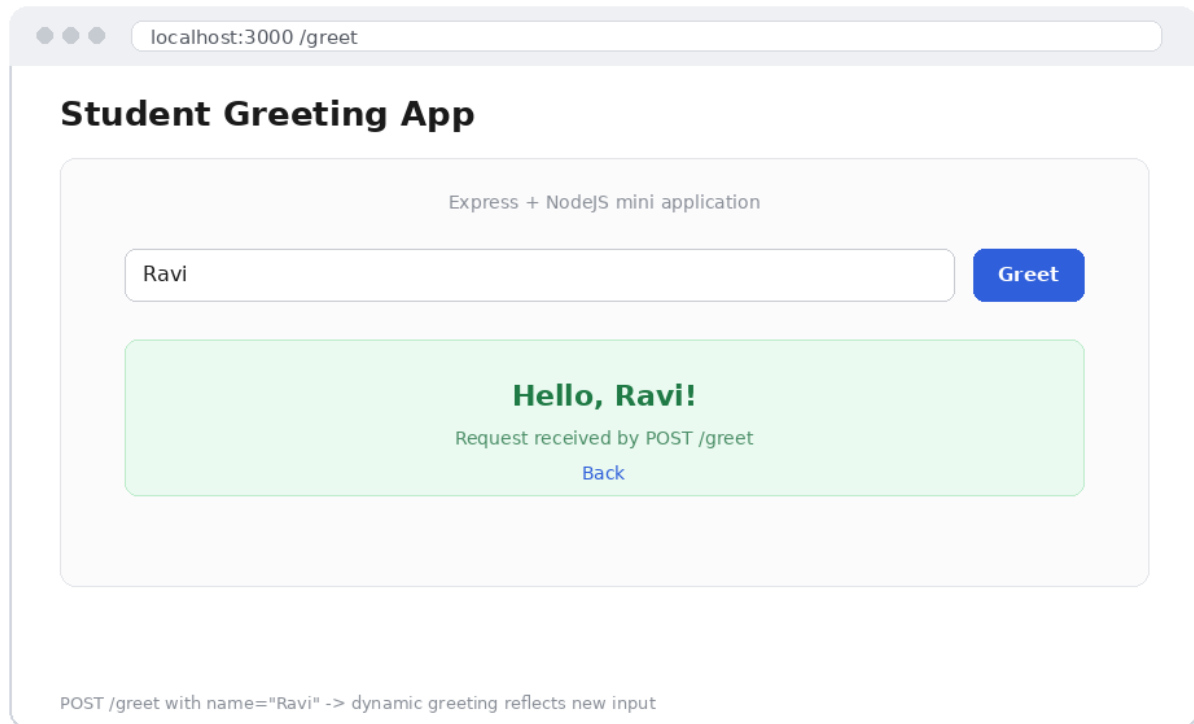
Result: PASS — output matches expected behaviour.

Test Case 3: POST /greet with name = "Ravi"

Request: POST /greet | Input: name = "Ravi"

Expected behaviour: Verifies the response is dynamic and not hardcoded — a different submitted name correctly changes the greeting to "Hello, Ravi!".

Output (Browser Screenshot):



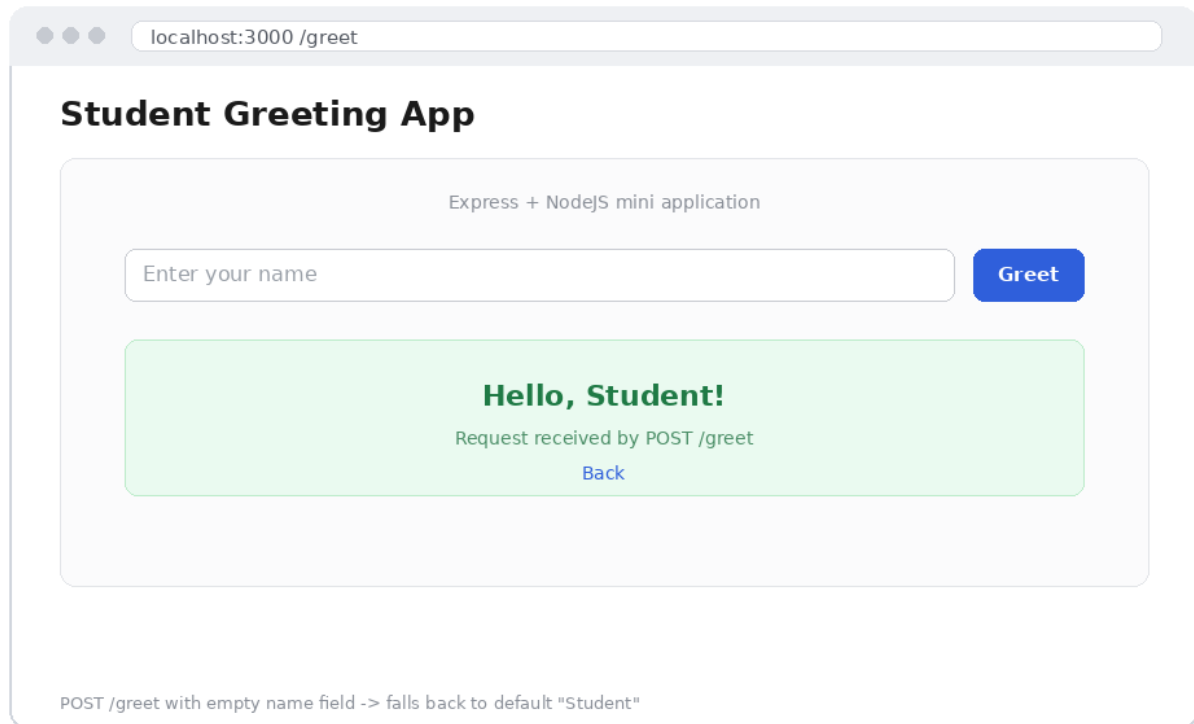
Result: PASS — output matches expected behaviour.

Test Case 4: POST /greet with an empty name field

Request: POST /greet | Input: name = "" (left blank)

Expected behaviour: Verifies the default-value fallback: since req.body.name is empty/falsy, the "|| 'Student'" fallback applies and the server responds with "Hello, Student!".

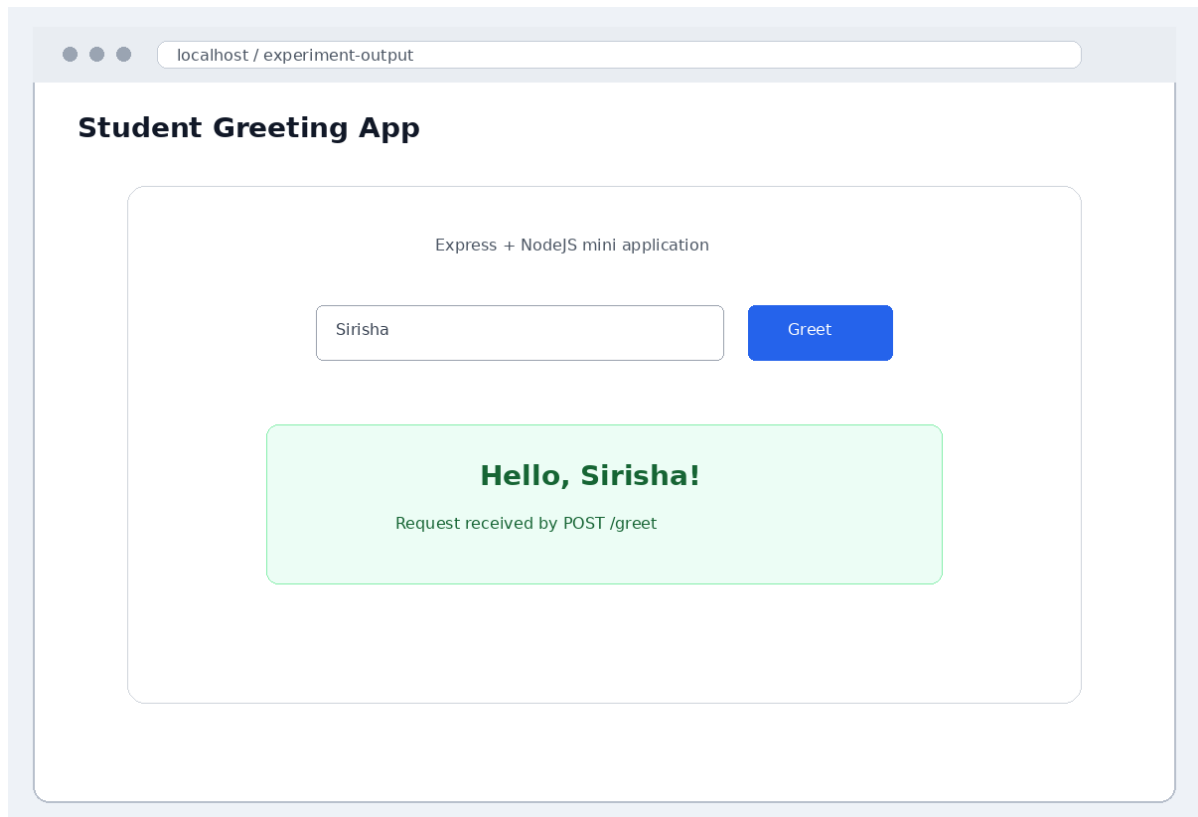
Output (Browser Screenshot):



Result: PASS — output matches expected behaviour.

9. OUTPUT

Browser Output (Screenshot):



10. RESULT

Thus, a mini web application was successfully developed using Express and NodeJS. The application accepts a name through a form, processes a POST request and returns a dynamic greeting response.